WorldLA: Bridging World Action Models and Efficient VLAs via Interaction Distillation and Coherent Replanning

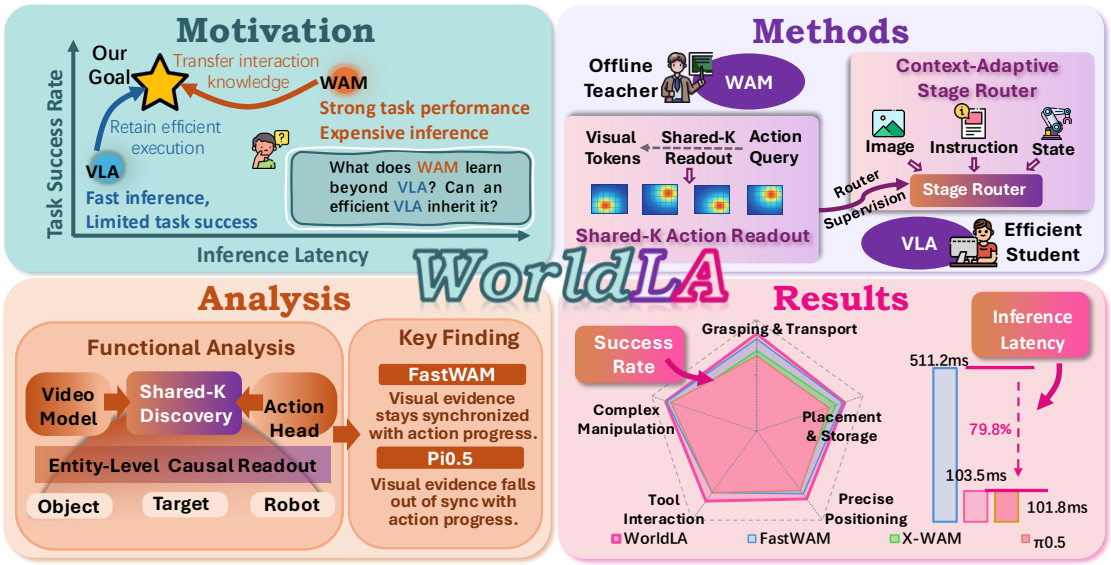


Figure 1. Overview of WorldLA. To combine WAM interaction capabilities with VLA efficiency, we analyze Fast-WAM's stage-dependent selection of visual evidence and use its shared-K distributions to supervise a lightweight stage router. The teacher is used only offline. WorldLA improves manipulation success on RoboTwin 2.0 while retaining inference latency close to the base VLA.

Abstract

World action models (WAMs) jointly model visual dynamics and actions, offering strong capabilities in complex manipulation at the computational cost of video networks. Vision-language-action models (VLAs) provide efficient inference but model the relationship between actions and scene changes less explicitly. To balance capability and efficiency, existing research separately pursues WAM acceleration and VLA enhancement. However, integrating these directions to combine WAM interaction capabilities with VLA execution efficiency remains underexplored. To this end, we compare Fast-WAM and $\pi_{0.5}$ in interaction perception and execution. In interaction perception, Fast-WAM's shared-key attention (shared-K) readout distributions consistently align with physical interaction stages, whereas $\pi_{0.5}$ lacks a comparable structure. In interaction execution, Fast-WAM exhibits greater interaction continuity across replanning boundaries. Based on these findings, we propose WorldLA. A lightweight stage router learns from offline shared-K distributions to inject stage-relevant information from the model's own visual features into corresponding action tokens. WorldLA also blends old and new plans to balance interaction continuity with observation-based correction. Fast-WAM is used only to construct offline supervision. Across 50 RoboTwin 2.0 tasks, WorldLA achieves an average success rate of 82.2%, outperforming $\pi_{0.5}$ at 70% and Fast-WAM at 77.8%. Its mean inference latency is 103.5 ms, close to $\pi_{0.5}$ at 101.8 ms and 79.8% lower than Fast-WAM. These results demonstrate that WAM interaction knowledge can improve VLA manipulation capabilities without deploying a video model.

1. Introduction

Vision-language-action models (VLAs) directly map visual observations, language instructions, and robot states to actions, enabling efficient and scalable control [2, 3, 14, 15, 23, 24, 26, 27, 31, 34, 42–44]. However, their training primarily focuses on action prediction, providing limited su-

pervision of action-induced environmental changes and potentially restricting their understanding of complex interaction dynamics. In contrast, world action models (WAMs) jointly learn future visual dynamics and actions, leveraging spatiotemporal representations from video models to improve manipulation [11, 16, 18, 37, 38]. These benefits persist even without future video generation at inference time [1, 38, 40, 45]. Nevertheless, large video networks and joint training incur substantial training costs and inference latency. To balance manipulation capability and execution efficiency, existing research reduces WAM costs by omitting future video generation or compressing models [38], or enhances VLAs through improved action experts and additional prediction objectives [19, 20, 23, 27, 43]. The former still incurs video-modeling overhead, while the latter relies on VLA training itself to learn relationships between dynamics and actions. How to integrate these two directions by transferring existing WAM interaction knowledge to efficient VLAs remains underexplored.

We compare Fast-WAM and $\pi_{0.5}$ through their internal information flow and closed-loop execution. Fast-WAM's shared-key attention (shared-K) connects action queries with visual features, and its routing distributions indicate which spatial regions provide information for different action positions. Our analysis shows that Fast-WAM's routing distributions consistently align with interaction stages, whereas $\pi_{0.5}$ lacks explicit modeling of the relationships among action positions, interaction stages, and spatial regions. During closed-loop execution, Fast-WAM continues ongoing interactions after replanning. Although $\pi_{0.5}$ can generate effective long action sequences, frequent replanning replaces effective but unexecuted actions in the sustained-contact tasks we analyze, and subsequent execution exhibits pronounced stagnation rather than continued interaction progress. These differences reveal two WAM capabilities: stage-conditioned modeling of action–scene relationships and preservation of interaction progress across replanning boundaries. Transferring these capabilities to efficient VLAs requires addressing two questions: (1) How can the stage-dependent relationships learned by WAMs be transferred to provide appropriate spatial information for different action stages? (2) How can VLAs use new observations to correct actions while preserving action continuity between old and new plans, allowing the ongoing operation to continue advancing after replanning?

To this end, we propose WorldLA (Fig. 1), which integrates WAM interaction capabilities into an efficient VLA. WorldLA trains a lightweight stage router using 20,000 offline shared-K routing supervision samples to extract stage-relevant information from its own features and inject it into corresponding action tokens. It also blends the unexecuted tail of the previous plan with the beginning of the new plan, balancing feedback correction with interaction continuity. Fast-WAM is used solely to construct offline supervision. Across the 50 demo_clean tasks in RoboTwin 2.0 [8], WorldLA achieves an average success rate of 82.2%, outperforming $\pi_{0.5}$ at 70% and Fast-WAM at 77.8%. Its average inference latency is 103.5 ms, close to $\pi_{0.5}$ and approximately $4.9\times$ faster than Fast-WAM. Our main contributions are as follows:

1. We identify two action-relevant differences between Fast-WAM and $\pi_{0.5}$ through fine-grained analysis: stage-aligned information selection and interaction continuity across replanning boundaries.
2. We propose WorldLA, a unified framework that transfers stage-dependent action–scene relations from WAMs to efficient VLAs and preserves interaction progress through continuity-preserving replanning.
3. We construct 20,000 action-stage routing supervision samples, converting internal information-reading behavior in a WAM into structured teacher signals for lightweight VLA learning.
4. WorldLA achieves 82.2% average success across the 50 demo_clean tasks in RoboTwin 2.0, surpassing the evaluated VLA and WAM baselines. It maintains VLA-level inference latency at 103.5 ms, achieving a $4.9\times$ speedup over Fast-WAM.

2. Related Work

2.1. Vision-Language-Action Models

Vision-language-action models map visual observations, language instructions, and robot states to actions, transferring vision-language knowledge to multitask robot control [2–4, 9, 14, 15, 17, 23, 24, 26, 27, 31, 34, 42–44, 47]. The π_0 family further uses flow-matching action experts to generate continuous action chunks and improves generalization through heterogeneous data and co-training [27, 31, 43]. These methods generate actions efficiently, but their training objectives primarily focus on action prediction. To improve understanding of environmental changes, several studies incorporate future visual state or world knowledge prediction into VLAs[28, 41, 48]. For example, DreamVLA predicts dynamic, spatial, and semantic information to support action generation [43]. These methods demonstrate the benefits of predictive supervision, while requiring visual dynamics to be learned within the student model. WorldLA extracts structured relationships relevant to action generation from a pretrained WAM, allowing the VLA to acquire existing interaction knowledge without introducing an additional future prediction branch.

2.2. World Action Models

World action models jointly model visual dynamics and robot actions, bringing video models' spatiotemporal representations into robot control. Cosmos-Policy, and

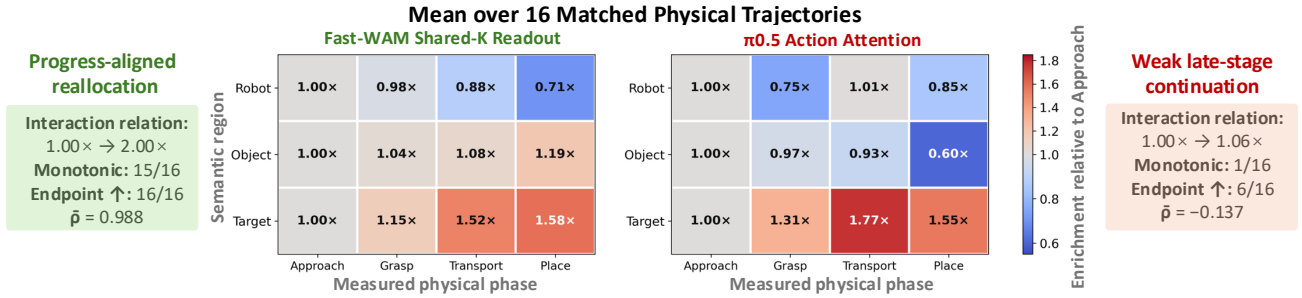


Figure 2. Stage-dependent visual readouts over 16 physically matched trajectories, normalized to the Approach stage. Fast-WAM increasingly prioritizes object and target information, while $\pi_{0.5}$ exhibits less consistent alignment with interaction progress.

Table 1. Post-replanning behavior in the microwave-opening task, evaluated over the next 10 environment steps. Opening progress is normalized.

Metric	$\pi_{0.5}$[24]	Fast-WAM[38]
Continued opening rate (%) ↑	20.29	65.25
Opening stagnation rate (%) ↓	67.85	18.50
Opening regression rate (%) ↓	2.56	0.29
Mean opening-progress gain ↑	0.0116	0.0361
Door-contact retention rate (%) ↑	99.50	99.90

DreamZero improve complex manipulation capabilities, but explicit future visual generation typically incurs substantial inference costs [7, 11, 16, 18, 21, 25, 30, 37, 38, 49]. To improve efficiency, Fast-WAM retains joint video-action training while skipping future video generation during deployment, demonstrating the benefits of learned internal representations [1, 38, 40, 45]. Faster-WAM further compresses the action module to reduce latency. However, these methods still incur the computational costs of video backbones and visual dynamics training. WorldLA identifies internal WAM mechanisms that directly support action generation and transfers them to an efficient VLA.

2.3. World-Model Knowledge Transfer

Knowledge distillation connects world modeling with efficient action generation[12, 22, 39]. THAW-VLA transfers cached visual features from a frozen world model to a compact VLA through cosine alignment, requiring no teacher at deployment [10]. However, its supervision excludes the action stream, pools each view into a single vector, and does not distinguish action positions or interaction stages. It emphasizes global visual representation transfer without analyzing the internal relations supporting action decisions. WorldLA instead analyzes Fast-WAM's shared-key attention (shared-K) to extract relations among action queries, spatial visual tokens, and relative action stages, providing action-conditioned, spatially fine-grained, and stage-dependent supervision.

3. Methods

3.1. What Do WAMs Learn Beyond VLAs?

Using Fast-WAM and $\pi_{0.5}$ as representative models, we examine which scene information action queries read at different interaction stages and whether ongoing operations continue advancing after replanning.

3.1.1. Stage-Dependent Information Selection

Fast-WAM's shared-key attention (shared-K) connects action queries with visual features from the video branch. We compare its readout distributions with action-query attention in $\pi_{0.5}$, examining only their relative changes across physical stages. We collect 16 sets of physically matched trajectories and divide them into four stages: Approach, Grasp, Transport, and Place. Stages are determined using end-effector positions, gripper states, object motion, and contact between the object and target. Visual tokens are assigned to three regions: Robot, Object, and Target. Let $E_{s,r}$ denote the area-corrected readout intensity for region r at stage s. We define the interaction ratio as

$$G_s = \frac{E_{s,\text{Object}} + E_{s,\text{Target}}}{E_{s,\text{Robot}}}. \tag{1}$$

Regional readout intensities and G_s are normalized relative to Approach within each trajectory before averaging across trajectories.

As shown in Fig. 3, Fast-WAM's Robot readout decreases to $0.71\times$ from Approach to Place, while its Object and Target readouts increase to $1.19\times$ and $1.58\times$, respectively. The interaction ratio rises from 1.00 to 2.00, with a mean stage correlation of $\bar{\rho} = 0.988$. Approaching requires determining the relative positions of the robot and object, whereas post-grasp manipulation increasingly requires information about the object and its spatial relationship to the placement target. Fast-WAM's readout shifts accordingly, from the robot region toward the object and target regions. In contrast, $\pi_{0.5}$'s Object readout falls to $0.60\times$ at Place, while its interaction ratio reaches only 1.06, with a mean

stage correlation of $\bar{\rho} = -0.137$. Its changes therefore lack consistent alignment with task progress.

These results show that Fast-WAM adjusts its selection of scene information with physical interaction progress, whereas $\pi_{0.5}$ lacks an equally consistent stage structure.

3.1.2. Continuity after Replanning

We conduct paired evaluations on a sustained-contact door-opening task using the same 50 seeds. At replanning boundaries with prior door contact or at least 2% opening progress, we measure the next 10-step gain $\Delta d^{(10)} = d_{t+10} - d_t$, where d_t denotes normalized opening progress. Continued progress, stagnation, and regression are defined as $\Delta d^{(10)} > 0.02$, $|\Delta d^{(10)}| \leq 0.005$, and $\Delta d^{(10)} < -0.005$, respectively. Fast-WAM achieves 96% task success, compared with 40% for $\pi_{0.5}$. As shown in Table 1, it exhibits higher continued progress (65.25% vs. 20.29%), lower stagnation (18.50% vs. 67.85%) and regression (0.29% vs. 2.56%), and a larger mean progress gain (0.0361 vs. 0.0116). Both models maintain contact at rates close to 100% (99.9% and 99.5%), yet $\pi_{0.5}$ frequently stalls after action updates, while Fast-WAM continues advancing the interaction. Together, stage-dependent information selection and sustained progress after replanning motivate WorldLA's stage-dynamic relation routing and continuity-preserving replanning, respectively.

3.2. Architecture

Given visual observations o_t, a language instruction l, and robot state z_t, WorldLA generates an action chunk of length H, denoted by $A_t = \{a_t^1, \ldots, a_t^H\}$.

As illustrated in Fig. 3, WorldLA comprises two components: (1)stage-dynamic relation routing and continuity-preserving replanning. The routing module learns which spatial regions to read at different action stages using offline shared-K distributions extracted from Fast-WAM. It injects the corresponding visual messages into the action expert. (2)The replanning module blends unexecuted actions from the previous plan with new actions generated from the latest observations, allowing feedback correction while advancing the ongoing operation. Fast-WAM is used only to construct offline supervision, while WorldLA independently performs action generation and closed-loop control.

3.2.1. Stage-Dynamic Relation Routing

Teacher Routing Construction. We first extract Fast-WAM's information-selection patterns at different action stages to supervise the student router. Fast-WAM's shared-K distributions record how strongly each action query reads from each visual position. We collect these distributions across all denoising steps and action positions, and divide action chunks into four stages according to relative position. This partition applies to both Fast-WAM's 32-step chunks and WorldLA's 50-step chunks, establishing correspondence between their stages.

For each stage, we aggregate the readout weights at each visual position across all action queries in that stage and all denoising steps. We then map the teacher's 120 visual positions to the student's 768 visual tokens and normalize the weights to sum to one. The resulting teacher distribution, denoted by q_s^T, specifies which positions stage s should read and their relative weights. Each sample contains four stage-specific distributions. Following this procedure, we construct 20,000 routing-supervision samples.

Stage Routing Prediction. WorldLA uses a lightweight router to predict a spatial readout distribution for each of the four action stages, conditioned on current observations, language instructions, and robot states. The router separately aggregates visual context and language-state context, then combines them with learnable stage embeddings to produce stage queries q_s. Each query represents the information needs of stage s in the current task. Meanwhile, each visual token is linearly projected into a visual key k_v with the same dimension as the query, representing the corresponding region's content for matching. The query-key dot product measures their compatibility and, together with a spatial prior, determines the readout weights:

$$P_{s,v} = \mathrm{Softmax}_v\left(b_{s,v} + \frac{q_s^\top k_v}{\tau\sqrt{d}}\right), \tag{2}$$

where $P_{s,v}$ is the weight assigned to visual position v at stage s, $b_{s,v}$ is a learnable stage-specific spatial prior, τ is a learnable temperature, and d is the query-key dimension. Task context adapts the readout distributions to the current scene, while stage embeddings allow different stages to develop distinct information-selection patterns.

Stage Information Injection. The predicted routing weights extract information from WorldLA's own visual features. Because visual and action features have different dimensions, a lightweight value adapter first maps each visual token from the 2048-dimensional visual space through a 128-dimensional bottleneck into the 1024-dimensional action feature space. The transformed features are then aggregated using the routing weights to produce a stage-specific visual message m_s. Each action token receives the message corresponding to its relative position within the action chunk:

$$\begin{aligned} m_s &= \sum_v P_{s,v}\,\mathrm{Adapter}(h_v^V), \\ \widetilde{h}_j^A &= h_j^A + \sigma(g_{s(j)})m_{s(j)}. \end{aligned} \tag{3}$$

Here, h_v^V is a visual token, h_j^A is an action token, and $s(j)$ denotes the stage assigned to action position j. The

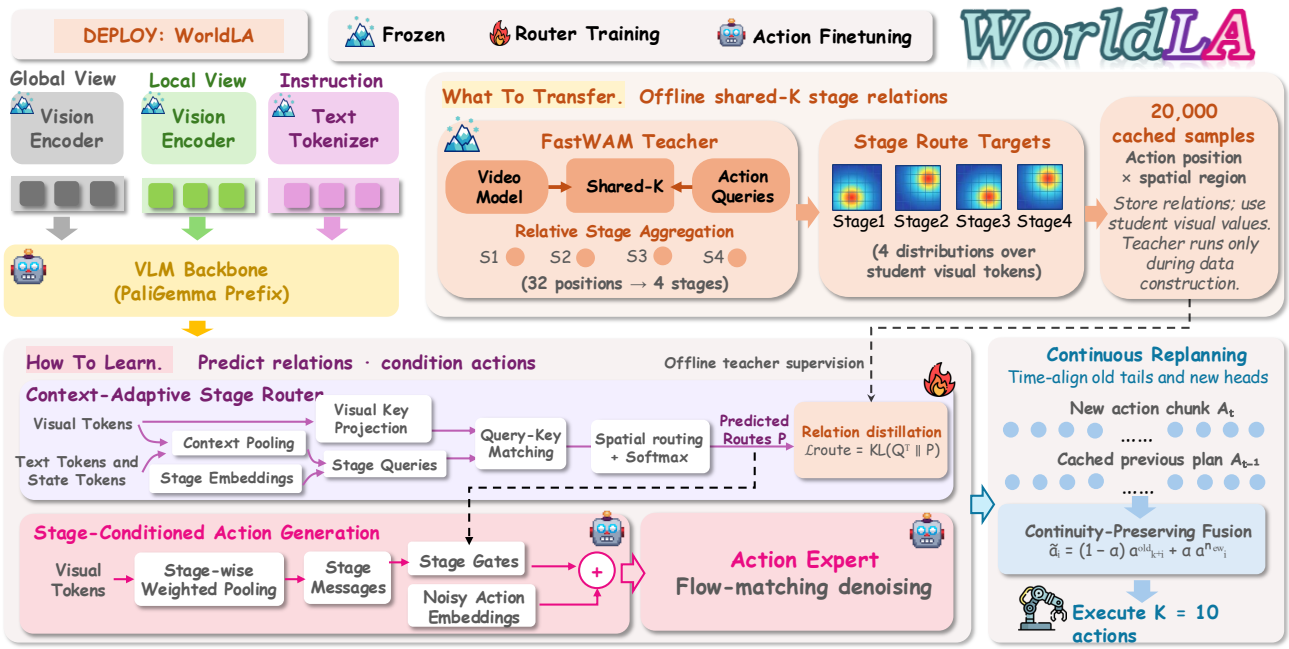


Figure 3. WorldLA framework. Offline shared key readout distributions from FastWAM supervise a stage router conditioned on task context. The router is then frozen to guide visual message injection during action adaptation. At deployment, old and new actions are aligned in time and blended to balance feedback correction and interaction continuity.

learnable gate $\sigma(g_s)$ controls the strength of message injection. Action tokens within the same stage share the same additional visual message, while different stages receive their respective messages. Fast-WAM thus supervises stage-specific information selection, while WorldLA constructs messages from its own visual features for action generation.

3.3. Continuity-Preserving Replanning

WorldLA generates 50-step action chunks and replans from the latest observation after executing $K = 10$ steps. To incorporate feedback while preserving useful motion trends from the previous plan, we blend its next K actions with the first K actions of the new plan:

$$\widetilde{a}_i = (1-\alpha_i)a^{\text{old}}_{K+i} + \alpha_i a^{\text{new}}_i, \qquad 1 \le i \le K, \tag{4}$$

where A^{old} and A^{new} denote the previous and new action chunks, respectively. We fix the new-plan weight at $\alpha_i = 0.25$ and the old-plan weight at 0.75, retaining existing motion trends while incorporating observation-based corrections. Arm joint commands are linearly interpolated, while gripper commands are selected discretely from the old or new plan. Actions outside the blending window are retained from the new plan. This procedure operates directly on generated action chunks and is decoupled from model inference. Stage-dynamic relation routing provides the action expert with stage-matched scene information, while continuity-preserving replanning maintains action continuity during plan updates. Together, they form WorldLA's training and closed-loop execution framework.

Stage-dynamic action–visual relation transfer provides the action expert with stage-matched information, while continuity-preserving replanning maintains interaction progress across planning boundaries. Together, they form WorldLA's training and closed-loop execution framework.

3.4. Two-Stage Training

Stage I: Relation Router Training. We freeze the VLA backbone and train the stage router using cached Fast-WAM shared-K distributions. Given visual, language, and robot state features, the router predicts four stage-specific spatial readout distributions, supervised through KL divergence:

$$\mathcal{L}_{\text{route}} = \frac{1}{S}\sum_{s=1}^{S} D_{\text{KL}}\left(q_s^T \| P_s\right), \tag{5}$$

where $S = 4$, and q_s^T and P_s denote the teacher and predicted distributions, respectively. Only router parameters are updated, enabling stage-specific information selection conditioned on the current task context.

Stage II: Routing-Guided Action Adaptation. We freeze the trained router and optimize the VLA's LoRA pa-

Table 2. Task-category and overall success rates (%) on RoboTwin 2.0. Category columns report C2C results, with task counts in parentheses. Bold indicates the best result among the displayed methods.

Method	Task categories (C2C)					Overall		
	Grasping & Transport (8)	Complex Manipulation (9)	Placement & Storage (15)	Precise Positioning (12)	Tool Interaction (6)	C2C ↑	C2R ↑	Avg. ↑
VLA								
starVLA[36]	51.75	45.44	47.73	38.17	54.83	46.52	3.16	24.84
Discrete Forcing[29]	56.50	61.11	60.80	57.33	60.67	59.32	11.20	35.26
GalaxeaVLA[13]	67.13	62.11	66.47	56.50	60.83	62.72	12.72	37.72
Xiaomi Robotics-0[6]	61.25	69.67	57.87	62.25	68.50	62.86	18.22	40.54
EventVLA[32]	51.63	75.56	65.47	68.33	64.00	65.58	15.74	40.66
Abot-M0[35]	47.88	68.56	62.07	58.42	39.67	57.40	30.36	43.88
X-VLA[46]	67.13	71.67	70.53	61.00	71.00	67.96	20.95	44.45
Spatial Forcing[17]	78.88	80.33	77.80	75.83	71.33	77.18	26.74	51.96
$\pi_{0.5}$[24]	68.13	79.44	69.33	71.67	62.17	70.66	46.00	58.33
WAM								
AHA-WAM[5]	53.00	76.89	66.13	60.08	64.50	64.32	3.20	33.76
FastWAM[38]	83.13	76.89	77.00	78.42	72.50	77.76	1.92	39.84
X-WAM[11]	76.13	66.44	69.47	62.67	83.00	69.98	25.84	47.91
4D-WAM[33]	83.13	82.33	77.73	**81.42**	**87.67**	81.50	41.80	61.65
WorldLA (ours)	**85.50**	**88.89**	**81.53**	78.00	77.83	**82.20**	**46.74**	**64.47**

Table 3. Online closed-loop timing on RoboTwin 2.0. Cycle time is averaged over successful episodes within each task and then equally across the 10 tasks.

Model	Mean inference latency (ms) ↓	Successful-episode cycle time (s) ↓
$\pi_{0.5}$[24]	101.8	25.68
Fast-WAM[38]	511.2	33.74
WorldLA	103.5	26.90

rameters, value adapter, and stage gates using robot demonstrations and the native flow-matching action loss. During each forward pass, stage-specific visual messages are aggregated according to the predicted routing weights and injected into corresponding action tokens through the gates, training the model to use this information for action generation. The value adapter's up-projection is zero-initialized, and the stage gates are initialized to small values, allowing the new message pathway to contribute gradually. Separating routing learning from action adaptation preserves the teacher-derived routing structure and reduces interference between the two training objectives.

4. Experiments

We evaluate WorldLA through five questions: **Q1:** Can WorldLA outperform existing VLA and WAM methods in task success rate? **Q2:** Can it improve manipulation performance while maintaining low inference latency? **Q3:** How do stage-dynamic relation routing and its Fast-WAM supervision contribute to performance? **Q4:** Does it reduce interaction stagnation and improve task success compared with standard replanning? We address these questions through performance comparisons, inference timing, core-component ablations, and generalization evaluations.

4.1. Benchmark Evaluation

4.1.1. Experimental Setup

We jointly train a single policy across 50 RoboTwin 2.0 tasks for 100K steps, without task-specific fine-tuning. Each task is evaluated over 100 trials, and we report the average success rate across all 50 tasks. We consider two evaluation settings. demo_clean uses scene configurations similar to those in the demonstrations to compare WorldLA with representative VLA and WAM methods. demo_randomized further randomizes object properties, initial positions, and scene layouts to assess robustness to visual and spatial distribution shifts. Both settings use the same 100K-step WorldLA checkpoint.

4.1.2. Benefits of WAM Knowledge Transfer

As shown in Table 2, WorldLA achieves an average success rate of 82.2% across the 50 demo_clean tasks, the highest among the compared methods. It exceeds the base model $\pi_{0.5}$ at 70.7% by 11.5 percentage points and its teacher Fast-WAM at 77.8% by 4.4 percentage points, while also outperforming X-WAM and AHA-WAM. These results support combining stage-dependent relational knowledge from WAMs with VLA action generation, allowing the student to surpass its teacher in overall task performance. Compared with $\pi_{0.5}$, WorldLA improves success on handover_block from 30% to 92% and on move_pillbottle_pad from 46%

Table 4. Component ablation on six RoboTwin 2.0 tasks with 100 trials per task. All variants use the $\pi_{0.5}$ backbone. Bold indicates the best result in each result column.

Variant	Components		Task success rate (%)						Avg. (%)
	Stage routing	Continuity replanning	Turn switch	Press stapler	Open microwave	Place can in basket	Place phone on stand	Move pill bottle to pad	
baseline	–	–	48.00	80.00	33.00	74.00	71.00	93.00	66.50
Routing only	✓	–	50.00	86.00	72.00	73.00	81.00	**99.00**	76.83
Replanning only	–	✓	**56.00**	82.00	65.00	70.00	73.00	88.00	72.33
WorldLA (full)	✓	✓	54.00	**87.00**	**81.00**	**82.00**	**82.00**	96.00	**80.33**



to 96%. These tasks require adapting information selection and sustaining interaction progress through localization, grasping, handover, or placement. The gains are consistent with WorldLA's design goals of stage-dependent relation routing and interaction continuity. To assess robustness to scene variations, we evaluate the same 100K-step checkpoint on demo_randomized without additional fine-tuning. WorldLA achieves an average success rate of 46.74% and outperforms $\pi_{0.5}$ on 27 of the 50 tasks. Success rates increase from 43% to 76% on stack_blocks_two, from 10% to 39% on move_can_pot, and from 47% to 74% on place_object_stand.

4.1.3. Inference Efficiency

We benchmark WorldLA, $\pi_{0.5}$, and Fast-WAM on 10 RoboTwin 2.0 tasks, with 50 episodes per model per task. Models run sequentially on the same A100 80GB GPU (batch size 1) under matched conditions. All use 10 denoising steps, replan every 10 actions, and retain native action chunks. After one warm-up episode per model and task, latency averages all policy calls, including failed episodes, from input processing to CPU action return, excluding environment execution. Cycle time spans observation acquisition through task success, including action blending but excluding loading and initialization. It is averaged over successful episodes within each task, then equally across tasks. As shown in Table 3, WorldLA achieves 103.5 ms latency, only 1.7 ms above $\pi_{0.5}$, with a $4.9\times$ speedup over Fast-WAM. Together with its task-performance gains, this demonstrates efficient manipulation without deploying a video model.

4.2. Core Component Ablation

4.2.1. Experimental Setup

To assess the individual contributions and combined effect of stage-dynamic relation routing and continuity-preserving replanning, we independently enable or disable the two components to form four configurations: the $\pi_{0.5}$ baseline, routing only, replanning only, and full WorldLA. Stage routing is supervised by Fast-WAM's shared-K distributions, while continuity-preserving replanning uses an action-blending weight of 0.25. Replanning modifies only inference-time action execution, leaving model training unchanged. We select six representative tasks requiring stage-dependent information selection or sustained interaction and evaluate each configuration over 100 trials per task.

4.2.2. Stage-Dynamic Relation Routing

As shown in Table 4, enabling stage routing alone increases the average success rate across the six tasks from 66.50% to 76.83%, a gain of 10.3 percentage points. With continuity-preserving replanning enabled, adding stage routing further improves success from 72.33% to 80.33%, a gain of 8.0 percentage points. Stage routing therefore provides gains under both replanning settings.

To further evaluate the routing-supervision design, we introduce controlled comparisons in Table 5. R0 is a VLA baseline without a router. R1 uses the same router architecture and parameter count as WorldLA but is trained with action loss only, testing whether the gains arise solely from additional parameters. R2 compares global visual feature distillation with spatial routing supervision to examine the role of the transferred content. R3 averages routes across stages to test whether different action stages require distinct spatial information. R4 preserves the spatial routing distributions but shuffles their correspondence with action stages, testing whether this information must be assigned to the correct stages. R5 uses WorldLA's stage-aligned routing supervision. R1 achieves an average success rate of 68.33%, compared with 66.50% for the baseline. With the same router architecture and parameter count, R5 reaches 76.83%, exceeding R1 by 8.5 percentage points. Thus, the gains cannot be attributed solely to additional parameters. R5 also outperforms global feature distillation at 69.67%, stage-averaged routing at 70.33%, and shuffled-stage routing at 68.50%. These results support the effectiveness of

Table 5. Routing-supervision ablation on 10 RoboTwin 2.0 tasks. Training indicators denote whether a component is trained at any stage of the full pipeline. SR denotes mean success rate (%).

ID	Configuration	Router supervision	Router trained	Action trained	Mean SR (%)
R0	baseline	None	–	✓	66.50
R1	Router w/o teacher	Action loss only	✓	✓	68.33
R2	Global feature distillation	Global WAM features	✓	✓	69.67
R3	Stage-averaged routing	Stage-averaged shared-K	✓	✓	70.33
R4	Shuffled-stage routing	Stage-shuffled shared-K	✓	✓	68.50
R5	WorldLA	Stage-aligned shared-K	✓	✓	76.83

Table 6. Sensitivity to the old-plan weight λ. Results are success rates (%). The new-plan weight is $1 - \lambda$. Shading marks the WorldLA default ($\lambda = 0.75$).

Old-plan weight λ	0.00	0.25	0.50	0.75	1.00
Stage-relation tasks	84.33	85.67	84.33	86.67	83.00
Sustained-interaction tasks	69.33	71.00	70.33	74.00	68.33
6-task average	76.83	78.33	77.33	80.33	75.67

spatial routing supervision and the importance of preserving distinct routing distributions across stages and their correct alignment with action stages.

4.2.3. Continuity-Preserving Replanning

As shown in Table 4, enabling continuity-preserving replanning alone increases the average success rate from 66.50% to 72.33%, a gain of 5.8 percentage points. With stage routing enabled, it further improves success from 76.83% to 80.33%, a gain of 3.5 percentage points. Replanning therefore provides benefits both with and without stage routing.

To evaluate the blending design and the choice of weight, we fix the WorldLA checkpoint and vary the old-plan weight $\lambda \in \{0, 0.25, 0.50, 0.75, 1.00\}$, assigning $1 - \lambda$ to the new plan. Within the blending window, $\lambda = 0$ uses only new actions, whereas $\lambda = 1$ retains only old actions. All settings use identical evaluation seeds and control configurations, requiring no additional training. As shown in Table 6, the default $\lambda = 0.75$ achieves the highest success rates among the tested weights on both stage-relation tasks (86.67%) and sustained-interaction tasks (74.00%), with a six-task average of 80.33%. Compared with using only new actions, the gains on the two task groups are 2.3 and 4.7 percentage points, respectively. The larger gain on sustained-interaction tasks is consistent with the goal of preserving interaction continuity. Retaining only old actions reduces the average to 75.67%, suggesting that observation-based corrections remain important. These results support blending previous and newly generated actions rather than relying exclusively on either plan. Together, stage routing and continuity-preserving replanning improve success over the baseline by 13.8 percentage points.

5. Conclusion

We presented WorldLA, which connects WAMs and efficient VLAs through interaction-knowledge transfer. Our analysis of Fast-WAM and $\pi_{0.5}$ identified two relevant differences: stage-conditioned action–visual relations and sustained interaction progress across replanning boundaries. These findings motivated a lightweight stage router trained with offline teacher supervision and a continuity-preserving replanning strategy. Together, they improve information selection and action continuity without requiring a video model during student training or deployment. On 50 RoboTwin 2.0 demo_clean tasks, WorldLA achieves an average success rate of 82.2%, exceeding both the base VLA and Fast-WAM. Its inference latency remains close to that of the base VLA while providing approximately a $4.9\times$ speedup over Fast-WAM. These results support targeted interaction-knowledge transfer as a practical way to connect visual dynamics modeling with efficient action generation. Future work will investigate robustness to broader scene variations and transfer to real-world manipulation.

References

[1] Arman Akbari, Ci Zhang, Arash Akbari, Lin Zhao, Yixiao Chen, Weiwei Chen, Xuan Zhang, Geng Yuan, and Yanzhi Wang. Flash-wam: Modality-aware distillation for world action models. *arXiv preprint arXiv:2606.05254*, 2026. 2, 3

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. 1

[4] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025. 2

[5] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, et al. Aha-wam: Asynchronous

horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026. 6

[6] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026. 6

[7] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024. 3

[8] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025. 2

[9] Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, et al. Internvla-m1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025. 2

[10] Trung Dao, Sankalp Yamsani, Jaden Park, Joohyung Kim, and Yong Jae Lee. Think like a world model, act like a vla: Distilling world-model representations into compact robot policies. *arXiv preprint arXiv:2609.24682*, 2026. 3

[11] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026. 2, 3, 6

[12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 3

[13] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxy open-world dataset and g0 dual-system vla model. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 20749–20756. IEEE, 2026. 6

[14] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. 1, 2

[15] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 1, 2

[16] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. 2, 3

[17] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, pages 132324–132345, 2026. 2, 6

[18] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026. 2, 3

[19] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024. 2

[20] Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025. 2

[21] Teli Ma, Jia Zheng, Zifan Wang, Chunli Jiang, Andy Cui, Junwei Liang, and Shuo Yang. Dit4dit: Jointly modeling video dynamics and actions for generalizable robot control. *arXiv preprint arXiv:2603.10448*, 2026. 3

[22] Wonpyo Park, Dongju Kim, Yan Lu, and Minsu Cho. Relational knowledge distillation. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3962–3971. IEEE, 2019. 3

[23] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, et al. $\pi^*_{0.6}$: a VLA That Learns From Experience. *arXiv preprint arXiv:2511.14759*, 2025. 1, 2

[24] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2, 3, 6

[25] Yichao Shen, Fangyun Wei, Zhiying Du, Yaobo Liang, Yan Lu, Jiaolong Yang, Nanning Zheng, and Baining Guo. Videovla: Video generators can be generalizable robot manipulators. *Advances in neural information processing systems*, 38:95597–95621, 2026. 3

[26] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 1, 2

[27] Wenxuan Song, Ziyang Zhou, Han Zhao, Jiayi Chen, Pengxiang Ding, Haodong Yan, Yuxin Huang, Feilong Tang, Donglin Wang, and Haoang Li. Reconvla: Reconstructive vision-language-action model as effective robot perceiver. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 18549–18557, 2026. 1, 2

[28] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 2

[29] Jingbo Wang, Wenxuan Song, Wenhao Yu, Han Zhao, Xi Wang, Jiayi Chen, Donglin Wang, Yan Wang, and Haoang Li. Discrete forcing: Infusing discrete guidance into continuous denoising for few-step action experts. *arXiv preprint arXiv:2609.39526*, 2026. 6

[30] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 3

[31] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026. 1, 2

[32] Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, et al. Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv preprint arXiv:2606.20092*, 2026. 6

[33] Lishan Yang, Wenxuan Song, Xi Wang, Pingyue Sheng, Zheng Fang, Ziyang Zhou, Junjie He, Haodong Yan, Jiayi Chen, Nan Sun, et al. 4d-wam: Infusing spatiotemporal awareness into world action models through trajectory fields. *arXiv preprint arXiv:2608.08023*, 2026. 6

[34] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. Efficientvla: Training-free acceleration and compression for vision-language-action models. *Advances in Neural Information Processing Systems*, 38:40891–40914, 2026. 1, 2

[35] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026. 6

[36] Jinhui Ye, Ning Gao, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen, Shu Liu, and Jiaya Jia. Starvla-α: Reducing complexity in vision-language-action systems. In *European Conference on Computer Vision*, pages 351–369. Springer, 2026. 6

[37] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 2, 3

[38] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 2, 3, 6

[39] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. *arXiv preprint arXiv:1612.03928*, 2016. 3

[40] Chushan Zhang, Jinguang Tong, Xuesong Li, Yikai Wang, and Hongdong Li. Keep the future, drop the rollout: Rift for world action models. *arXiv preprint arXiv:2608.11521*, 2026. 2, 3

[41] Jianke Zhang, Yanjiang Guo, Yucheng Hu, Xiaoyu Chen, Xiang Zhu, and Jianyu Chen. Up-vla: A unified understanding and prediction model for embodied agent. *arXiv preprint arXiv:2501.18867*, 2025. 2

[42] Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, Xiaowei Chi, Gaole Dai, Li Du, Dan Wang, Yuan Du, and Shanghang Zhang. Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 18764–18772, 2026. 1, 2

[43] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, et al. Dreamvla: a vision-language-action model dreamed with comprehensive world knowledge. *Advances in Neural Information Processing Systems*, 38:24195–24228, 2026. 2

[44] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1702–1713. IEEE, 2025. 1, 2

[45] Weisen Zhao, Lam Nguyen, Zhicong Lu, and Yuzhang Shang. C ̓3 ache: Accelerating world action models with cross inference chunk cache. *arXiv preprint arXiv:2606.08962*, 2026. 2, 3

[46] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, pages 60580–60606, 2026. 6

[47] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, pages 54277–54296, 2025. 2

[48] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loic Magne, et al. Flare: Robot learning with implicit world modeling. *arXiv preprint arXiv:2505.15659*, 2025. 2

[49] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025. 3